



Site web collaboratif de recettes de cuisine

Arthur Lagenebre — Certification RNCP Développeur Web

[À COMPLÉTER : date de soutenance]

Devant : [À COMPLÉTER : noms des jurés]

Le pitch en 30 secondes

Problème. Les sites de recettes existants sont saturés de publicité et ne valorisent pas la communauté.

Solution. Une plateforme où les utilisateurs publient, commentent et sauvegardent des recettes — avec une **modération transparente** et une **recherche par ingrédients**.

Preuve. Une application **full-stack** que je vais vous présenter, déployée en ligne, dont **chaque ligne de back-end est écrite à la main**.

Plan de la présentation

1. **Cahier des charges** et démarche (2 min)
2. **Bloc 1** — Front statique, responsive, accessibilité (2 min)
3. **Bloc 2** — Back-end *from scratch* (8 min) — le cœur du projet
4. **Bloc 3** — Front Angular qui consomme l'API (3 min)
5. **Démo en ligne** (2 min)
6. **Améliorations** et perspectives (1 min)
7. **Vos questions**

Cahier des charges — 3 blocs imposés

Bloc	Exigence clé	Mon choix
1. Front statique	HTML/CSS/JS responsive + accessible WCAG	Angular sert aussi pour le Bloc 1 (✅ vu avec le formateur)
2. Back-end	From scratch — sans framework ni librairie prédéfinie + POO + MySQL	Node.js + Express 5 + MySQL via <code>mysql2</code>
3. Framework	React / Angular / Symfony / Laravel / Flask	Angular 21 — SSR, Signals, standalone

Contrainte transverse : tout doit être documenté + déployé en ligne + défendable à l'oral.

Démarche de travail

Méthodologie incrémentale, traçable, testable.

- **Git Flow simplifié** : branches `feat/*`, `fix/*`, `docs/*` + Conventional Commits
- **3 repos séparés** : `backend`, `frontend`, `documentation` — découplage assumé
- **Tests dès le début** : ~40 fichiers de tests sur le backend, lancés avec le test runner natif `node:test` (pas de Jest)
- **ADRs** — chaque décision structurante est tracée dans un document court (`Architecture Decision Record`)
- **UML** — 5 diagrammes avant d'écrire la première ligne d'API

Bloc 1 — Responsive & accessibilité

Approche : Angular sert de couche de présentation pour les 2 blocs (1 et 3). Le front respecte les exigences du Bloc 1 *même si* le rendu est piloté par Angular.

- **Responsive** : Bootstrap 5 + media queries personnalisées, testé sur 3 breakpoints (mobile 375px, tablette 768px, desktop 1280px)
- **Accessibilité** : rôles ARIA, contrastes $\geq 4.5:1$, navigation clavier complète, `<label>` sur tous les inputs
- **Tests** : BrowserStack sur Chrome / Firefox / Safari / Edge

Score Lighthouse

[Insérer ici une capture d'écran Lighthouse à 4 jauges]

Métrique	Score
Performance	[À COMPLÉTER]
Accessibility	[À COMPLÉTER — viser ≥ 95]
Best practices	[À COMPLÉTER]
SEO	[À COMPLÉTER]

Outils utilisés : Lighthouse, axe DevTools, BrowserStack

Conformité WCAG / RGAA

Critère WCAG	Implémentation
1.4.3 Contraste minimum (AA)	Palette validée à 4.5:1 sur fond clair, 7:1 sur fond foncé
2.1.1 Tout au clavier	<code>tabindex</code> , <code>:focus-visible</code> , skip links
2.4.4 Liens explicites	Pas de "cliquez ici" — chaque <code><a></code> a un libellé porteur de sens
3.3.2 Étiquettes de formulaire	Chaque input a un <code><label for=""></code> ou <code>aria-label</code>
4.1.2 Nom/rôle/valeur	Rôles ARIA sur composants custom (dialog, menu, alert)

Tests : navigation au clavier seul + lecteur d'écran NVDA sur la home et le formulaire de soumission.

Bloc 2 — Back-end *from scratch*

Le cœur du projet — et de cette présentation

« From scratch » — ma définition

Le cahier des charges interdit les **frameworks et librairies prédéfinies**. J'ai tracé la frontière comme ceci :

❌ Interdit (pas utilisé)	✅ Autorisé (utilisé)
ORM (Sequelize, Prisma, TypeORM)	Driver MySQL (<code>mysql2</code>) — bas-niveau
Framework de DI (NestJS, InversifyJS)	Injection manuelle dans <code>app.ts</code>
Scaffolding (<code>express-generator</code> , <code>nest new</code>)	Architecture conçue à la main
Validation auto (Joi, class-validator)	DTOs avec validation maison
Auth packagée (Passport, Auth0)	JWT + bcrypt assemblés à la main

Express et le driver MySQL sont des **interfaces** vers HTTP et la BDD — pas du métier.

📄 Détail : [soutenance/backend/adr/00-narration-from-scratch.md](#)

Architecture en couches

```
graph LR
  A[Client HTTP] --> B[Express<br/>Router]
  B --> C[Controller]
  C --> D[Service<br/>logique métier]
  D --> E[Repository<br/>interface]
  E --> F[RepositoryImpl<br/>SQL]
  F --> G[(MySQL)]

  style D fill:#e74c3c,color:#fff
  style E fill:#3498db,color:#fff
```

- **Controller** : parsing HTTP → DTO → appel service → sérialisation réponse
- **Service** : règles métier (validation, autorisations, orchestration)
- **Repository** : pattern interface + implémentation pour découpler le SQL

Le service ne sait pas que MySQL existe. Il parle à une interface.  Testable,  remplaçable.

Pattern Repository — exemple concret

Interface (`IRecipeRepository`):

```
interface IRecipeRepository {  
  findById(id: number): Promise<Recipe | null>;  
  create(data: CreateRecipeDto): Promise<Recipe>;  
  softDelete(id: number, by: number): Promise<void>;  
}
```

Implémentation (`RecipeRepository`):

```
class RecipeRepository implements IRecipeRepository {  
  constructor(private pool: Pool) {}  
  async findById(id: number) {  
    const [rows] = await this.pool.query('SELECT * FROM recipes WHERE id = ?', [id]);  
    return rows[0] ? this.mapper.toDomain(rows[0]) : null;  
  }  
  // ...  
}
```

Schéma BDD — vue domaine

[Insérer ici le diagramme `_draft_uml/01-diagramme-classes-domaine.md` (vue social ou complète)]

Tables clés :

- `users` (auth + profil + rôle ENUM `'user' | 'admin'`)
- `recipes` (titre, slug, contenu JSON, statut modération)
- `comments`, `favorites`, `categories`, `recipe_categories`

Choix de modélisation :

- Soft-delete partout (`deleted_at` `TIMESTAMP` `NULL`) — 📄 ADR-004
- Slug en 2 phases (réservation + finalisation) — 📄 ADR-005
- JSON pour ingrédients + étapes — schéma libre, validation côté DTO

Authentication — JWT + cookie HttpOnly

Flow d'inscription puis connexion :

```
sequenceDiagram
    participant C as Client
    participant A as AuthController
    participant S as AuthService
    participant R as UserRepo
    participant DB as MySQL

    C->>A: POST /auth/login {email, pwd}
    A->>S: login(dto)
    S->>R: findByEmail(email)
    R->>DB: SELECT
    DB-->>R: row
    R-->>S: User
    S->>S: bcrypt.compare(pwd, hash)
    S->>S: jwt.sign({id, role}, secret, 7j)
    S-->>A: token
    A-->>C: Set-Cookie: token=...; HttpOnly; Secure; SameSite=Strict
```

Sécurité — défense en profondeur

Menace OWASP	Parade implémentée
A01 Broken Access Control	Middleware <code>requireAuth</code> + <code>requireRole('admin')</code> sur les routes sensibles
A02 Cryptographic Failures	<code>bcrypt</code> 12 rounds pour les mots de passe ; HTTPS en prod
A03 Injection (SQL)	Requêtes paramétrées partout — jamais de concat string
A05 Security Misconfig	<code>helmet</code> désactivé volontairement (from-scratch), headers réécrits à la main
A07 Identification & Auth Failures	Rate limit sur <code>/auth/login</code> (5/min/IP) ; pas d'enum de comptes

Validation côté entrée : tous les payloads passent par un DTO avec validation maison avant d'atteindre le service.

Tests — ~40 fichiers, `node:test` natif

```
import { test } from 'node:test';
import assert from 'node:assert';

test('RecipeService.create refuse un titre vide', async () => {
  const repoMock = { create: async () => { throw new Error('should not be called'); } };
  const service = new RecipeService(repoMock as any);
  await assert.rejects(
    () => service.create({ title: '', /* ... */ }),
    /title is required/
  );
});
```

Couverture :

- Services métier : validation, autorisation, orchestration
- Middlewares : auth, role, rate-limit
- DTOs : règles de validation
- Mappers : domaine ➡ persistance

Modération — soft-delete + log

Règles métier :

1. Un commentaire signalé reste **visible** mais marqué `pending_review`.
2. L'admin **n'efface pas** — il **masque** (`deleted_at = NOW()`) et **trace** dans `moderation_log`.
3. L'utilisateur peut être notifié si sa recette est modérée.

```
INSERT INTO moderation_log  
  (target_type, target_id, action, by_user, reason, at)  
VALUES ('recipe', ?, 'hide', ?, ?, NOW());
```

Pourquoi pas un DELETE pur ? Traçabilité (RGPD, audit) + réversibilité (erreur de modération).

 ADR-004.

Cas d'utilisation — vue d'ensemble

[Insérer ici le diagramme `_draft_uml/03-diagramme-cas-utilisation.md`]

Acteurs :

-  **Visiteur** : consulter, rechercher
-  **Utilisateur authentifié** : publier, commenter, favoris
-  **Admin** : modérer, masquer, restaurer

Frontières :

- Inscription = ouverte
- Publication = authentifié
- Modération = rôle admin

A Bloc 3 — Angular 21

Front-end qui consomme l'API Bloc 2

Pourquoi Angular 21

Critère	Décision
Signals	Réactivité fine, pas de Zone.js — perf garantie sans surveillance manuelle
Standalone components	Plus de NgModule — chaque composant déclare ses dépendances → lazy load granulaire
SSR (Server-Side Rendering)	SEO + perf perçue (first contentful paint < 1s)
TypeScript strict	<code>strict: true</code> partout — null-safety, exhaustive checks
Maturité écosystème	Angular CLI, DevTools, schematics — outillage solide pour 1 dev

Le contraste assumé : Bloc 2 = artisanal, Bloc 3 = industriel. C'est l'esprit du cahier des charges.

Architecture front

```
src/app/  
├── core/                # services singletons (AuthService, ApiService)  
│   ├── guards/         # AuthGuard, AdminGuard  
│   └── interceptors/    # AuthInterceptor (cookie auto), ErrorInterceptor  
├── shared/             # composants réutilisables (RecipeCard, Modal)  
├── features/             
│   ├── recipes/        # liste, détail, formulaire  
│   ├── auth/           # login, register, profile  
│   └── moderation/     # back-office admin (lazy-loaded)  
└── app.routes.ts       # routes top-level
```

State : `signal()` au niveau service. Pas de NgRx — overkill pour ce projet.

Communication API : 1 service par feature, retourne `Observable<T>` (HttpClient) **OU** `Signal<T>` via `toSignal()`.

Sécurité côté front

Préoccupation	Mesure
XSS	Bindings Angular échappent par défaut ; pas de <code>innerHTML</code> non sanitisé
Vol de session	Cookie HttpOnly → inaccessible au JS → token non volable par script tiers
CSRF	<code>SameSite=Strict</code> sur le cookie + header custom <code>X-Requested-With</code> côté Angular
CSP	Header <code>Content-Security-Policy</code> côté serveur, pas d'inline scripts dans Angular build
Erreurs leakées	<code>ErrorInterceptor</code> global qui n'affiche jamais le stack au user, log côté serveur

Tests : Karma + Jasmine sur les composants critiques (formulaire d'inscription, garde admin).

Performance & accessibilité

[Insérer ici capture Lighthouse Angular ou Angular DevTools]

- **First Contentful Paint** : [À COMPLÉTER] grâce au SSR
- **Bundle size** : [À COMPLÉTER] — lazy load des routes back-office
- **Hydratation partielle** : Angular 21 ne ré-hydrate que ce qui est interactif
- **A11y Angular** : `cdkA11yModule`, focus trap dans les modales, annonces ARIA live

Le pari : SSR + Signals donne un site qui *ressemble* à du statique mais qui est dynamique. Bon pour les utilisateurs + bon pour Google.

Démo en ligne

[URL de prod à compléter]

Scénario de démo

1. Consultation publique (30 s)

→ Home → recherche par ingrédient → détail d'une recette

2. Inscription + publication (45 s)

→ Inscription nouveau compte → formulaire de soumission → publication

3. Modération (30 s)

→ Bascule sur le compte admin → file de modération → masquer un commentaire signalé

4. Responsive en direct (15 s)

→ Devtools → vue mobile → vérifier que le menu burger marche

Total démo : ~2 minutes. Précis, chronométré, répété 3 fois la veille.



Détail : <soutenance/demo/scenario-demo.md>

Améliorations envisagées

Domaine	Amélioration	Effort
Auth	Refresh tokens + rotation	1 j
Perf	Cache Redis sur les pages recettes les plus vues	2 j
Modération	Pré-filtre auto par modèle ML (toxic-comment)	5 j
Recherche	Moteur full-text dédié (Meilisearch / Typesense)	3 j
Observabilité	Logs structurés (pino) + Prometheus + Grafana	2 j
CI/CD	Tests d'intégration Docker + déploiement auto	3 j

Ces points ne sont pas des manques — ils sont **identifiés**, **chiffrés** et **priorisés**. C'est la différence entre une v1 et un produit.

Ce que ce projet m'a appris

- **Faire des compromis conscients** — chaque ADR est un choix assumé, pas un défaut subi.
- **Le code à la main n'est pas du code primitif** — c'est du code dont on comprend chaque ligne.
- **Tester tôt rend libre** — refactoriser sans peur grâce à 40 tests qui tournent en 2 secondes.
- **Documenter en marchant** — les ADRs ont été écrites au moment de la décision, pas après.
- **Savoir ce qu'on ne sait pas** — j'ai identifié des angles morts (refresh tokens, observabilité, ML) — la prochaine étape c'est de les combler.



Vos questions ?

Code & docs :

- Backend → `github.com/[À COMPLÉTER]/recipe-shelter-backend`
- Frontend → `github.com/[À COMPLÉTER]/recipe-shelter-frontend`
- Documentation → `github.com/[À COMPLÉTER]/recipe-shelter-documentation`

Contact :

- Arthur Lagenebre — [À COMPLÉTER : email / LinkedIn]